

C#.NET

Abstract Classes



Mr. Bangar Raju

NARESH
technologies

Understanding Abstract Classes and Methods in C#.NET

By Albert Harmon

Published December 27, 2016

🕒 4 min read

[Education](#)



TLDR This blog post explains the concepts of abstract classes and methods in C#.NET, detailing their definitions, usage, and the mandatory implementation requirements for child classes. It also highlights the differences between abstract methods and

method overriding, providing practical examples for better understanding.

In the realm of object-oriented programming, abstract classes and methods play a crucial role in defining a blueprint for other classes. This blog post aims to clarify what abstract classes and methods are, their significance in application development, and how they differ from regular methods and classes.

What is an Abstract Method?

An abstract method is defined as a method that lacks a body. It consists solely of a declaration, meaning it does not provide any implementation. For example:

```
public abstract void Add(int x, int y);
```

In this case, the method `Add` is declared as abstract, which means it must be implemented in any derived class. To declare a method as abstract, the `abstract` modifier must be explicitly used.

What is an Abstract Class?

An abstract class is a class that contains one or more abstract methods. If a class includes any abstract members, it must also be declared as

abstract. For instance:

```
public abstract class SomeMath
```

This class can contain both abstract methods and non-abstract methods. However, it cannot be instantiated directly. Instead, it serves as a base class for other classes that will implement its abstract methods.

The Role of Child Classes

When a class inherits from an abstract class, it is required to implement all abstract methods defined in the parent class. This requirement ensures that the child class adheres to the structure imposed by the abstract class. If a child class fails to implement these methods, it will not compile.

Example of Implementation

Consider the following abstract class:

```
public abstract class ABSParent
{
    public abstract void Add(int x, int y);
    public void Subtract(int x, int y)
```

```
{  
    Console.WriteLine(x - y);  
}  
}
```

In this example, `ABSParent` has one abstract method `Add` and one non-abstract method `Subtract`. A child class must implement the `Add` method to be instantiated:

```
public class ABSChild : ABSParent  
{  
    public override void Add(int x, int y)  
    {  
        Console.WriteLine(x + y);  
    }  
}
```

Differences Between Abstract Methods and Method Overriding

While both abstract methods and method overriding involve inheritance, they serve different purposes:

- **Abstract Methods:** Must be implemented in the child class. The implementation is mandatory, and the child class cannot escape

this requirement.

- **Method Overriding:** Involves redefining a method in a derived class that was already implemented in the base class. This is optional; the child class can choose whether or not to override the method.

Example of Method Overriding

Consider a base class with a virtual method:

```
public class BaseClass
{
    public virtual void Show()
    {
        Console.WriteLine("Base Show");
    }
}
```

A derived class can override this method:

```
public class DerivedClass : BaseClass
{
    public override void Show()
    {
        Console.WriteLine("Derived Show");
    }
}
```

```
}  
}
```

In this case, overriding is optional, and the derived class can choose to use the base class implementation.

Restrictions on Abstract Classes

An important restriction to note is that you cannot create an instance of an abstract class. For example:

```
ABSParent p = new ABSParent(); // This will cause a compilatio
```



However, you can create a reference of an abstract class using a child class instance:

```
ABSChild c = new ABSChild();  
ABSParent p = c; // This is valid
```

Conclusion

In summary, abstract classes and methods are fundamental concepts in C#.NET that enforce a structure on derived classes. They ensure that certain methods are implemented, promoting a consistent interface across different implementations. Understanding these concepts is essential for effective object-oriented programming in C#.NET.

By adhering to the rules of abstract classes and methods, developers can create robust and maintainable code that leverages the power of inheritance and polymorphism.